

Codexa Proof and Runtime Hardening

PR summary for `codex/enhance-features` against `main`.

- Base: `834bd5e` (v0.17.1)
- Final source head before this evidence refresh: `01d204f`
- Branch delta at that head: 88 files, 6,413 insertions, 598 deletions
- Commits before this evidence refresh: 6 Conventional Commits
- `d04c5c2` — feat: harden Codexa proof and runtime workflows
- `5ef56da` — fix: close human workflow and MCP interop gaps
- `46104e9` — fix(proof): scope Cypress verification credit
- `5afb683` — docs: add enhancement PR summary
- `9864579` — docs: correct final gate evidence
- `01d204f` — fix: close PR review and artifact boundary gaps

Executive Outcome

The sweep concentrated on the highest-ROI trust boundaries: proof authority, verification credit, durable local state, managed-artifact boundaries, real multi-language workflows, MCP coexistence, hot-path bounds, stable CI fan-out, and release supply-chain integrity. The result is a more deterministic completion system that fails closed when evidence or local provenance is ambiguous, survives concurrent and interrupted writers, and remains useful in ordinary Python, JavaScript, TypeScript, monorepo, nested-repository, and multi-MCP use.

The design follows large-scale functional-core principles: explicit state transitions, pure bounded matchers and classifiers, immutable evidence inputs, small policy modules, deterministic identities, and side effects isolated at validated persistence and process boundaries.

Problems Addressed

Codexa's completion authority could previously be weakened by stale or ambiguous proof, overly broad command credit, concurrent local receipt writers, damaged or redirected session state, and workspace identities that did not bind every raw byte. Verification recipes did not cover enough real-world runner and package layouts. MCP cleanup could also mistake an unrelated Graphify server for Codexa when its executable path happened to contain the word `codexa`, while release automation still depended on floating action and publisher downloads. The final boundary sweep additionally found that generated files, parse caches, MCP resources, and outcome receipts needed one shared protection model against directory redirection, symlink/hardlink substitution, unsafe portable names, unbounded reads, partial writes, and repository-authored cache poisoning.

Highest-ROI Adjustments

1. Evidence and proof authority

- Reconstructs required test, workflow, and dependency checks from saved task scope instead of allowing incomplete completion claims.

- Preserves safe repository-bound command handoffs while keeping previews, reports, artifacts, failures, and waivers semantically distinct.
- Binds review authority to exact raw workspace content across dirty-to-commit transitions, nested repositories, line-ending normalization, file-mode changes, and status-hidden edits.
- Credits only bounded, executable test scopes through command-classifier v7. Python unittest discovery, Node test globs, Playwright, and Cypress are supported; filtered, unsafe, unmatched, ambiguous, and outside-repository invocations fail closed.
- Runner provenance now travels with coverage, ledger, compaction, and required checks. Cypress recommendations accept only Cypress evidence; generic JavaScript, Jest, Vitest, Playwright, legacy, broad, fallback, and stale coverage cannot receive Cypress credit. One explicit indexed `--spec` remains target-only evidence, including conventional `*.cy.ts/js` filenames.

2. Real workflow coverage

- Generates runner- and package-aware candidate verification commands.
- Completes broad post-edit coverage in bounded passes.
- Serializes concurrent outcome-pointer and hook-journal publication with collision-safe identities.
- Preserves exact proof context through commit, reopen, and re-index flows.

3. Durable session state

- Reconciles the write-ahead event log with the materialized session store.
- Rejects redirected, cross-session, and non-regular managed state.
- Recovers from interrupted log tails without trusting damaged evidence.
- Maps non-portable session identifiers to deterministic filesystem-safe names.

4. Managed-artifact integrity

- Centralizes component-wise repository containment and non-symlink directory validation for all generated artifacts.
- Publishes through exclusive random staging files and atomic rename, verifies single-link regular-file identity, loops across partial writes, and bounds reads and directory-scan concurrency.
- Excludes redirected, hardlinked, malformed, oversized, and non-portable module/MCP/outcome entries from both direct reads and advertised listings.
- Treats the parse cache as local derived state only: a force-tracked cache or any failed Git provenance inspection is neither read nor overwritten, so a repository author cannot inject valid-hash forged parse facts.
- Restricts index publication to the managed `.codex/codebase` boundary and safely recovers the newest valid backup without following external state.

5. Bounded retrieval and portable artifacts

- Uses bounded compiled matcher caches with parity between search and retrieval scoring.
- Applies shared raw-search limits and preserves the `git grep` fallback path.
- Gives generated module artifacts deterministic, collision-safe filenames even when names collapse to the same slug or run on case-insensitive filesystems.

6. MCP, CI, and release integrity

- Preserves unrelated Graphify MCP configuration during Codexa initialization.
- Compiles the main development gate once instead of rebuilding for each test phase.

- Caps process-heavy Vitest file fan-out at two workers and gives the escaped- pipe fixture a realistic startup window, eliminating scheduler-induced false deadline failures without relaxing production timeout assertions.
- Pins GitHub Actions to immutable commits.
- Pins MCP Publisher by version and SHA-256 before extraction.

Verification

Deterministic release gate

The exact clean source head `01d204f` completed the full `npm run security:check` gate. Compile/build, source hygiene, privacy, and all 117 Claude command/hook integration smokes passed. The bounded two-worker built suite passed 103/103 files with 1,206 tests passing and 1 intentional platform skip; no timing waiver or isolated-rerun substitution is used for the final result.

- npm audit reported 0 vulnerabilities.
- The clean one-commit public snapshot, package hygiene, plugin hygiene, and startup-context budget passed.
- The packed npm tarball installed into a fresh consumer and passed all 31 smoke checks, including CLI, review, hooks, direct MCP startup, plugin MCP startup, Claude Code launcher startup, and workspace-focus routing.
- The eval gate passed 21/21 scenarios with score 1 and no raw-rg wins.
- The exact CI-scaled benchmark passed 11/12 metrics locally; one `cli.repo_map` p95 sample was 4.725 seconds against the 4.5-second shared-runner ceiling while its median was 0.960 seconds. The GitHub benchmark job must pass before merge.
- `git diff --check` passed.
- Current-snapshot privacy checks passed. `privacy:history` reports only four pre-existing commits already reachable from `origin/main`; published history is not rewritten as part of this change.

Human workflow simulations

Four disposable repositories were changed, tested, committed, and reopened as a developer would use them. They were then re-indexed and strict-reopened again with final-head `01d204f`:

- Python `src/` layout: 5 unittest cases passed with the repository-declared `PYTHONPATH=src` recipe.
- JavaScript monorepo: 3 Node test cases passed across workspace packages.
- TypeScript package: 4 Vitest cases passed after a behavior change.
- JavaScript service with Graphify: 6 Node test cases passed after pricing and order-validation changes.

The final rerun produced fresh, identity-matched indexes with zero parser errors for all four repositories. Earlier commit/reopen phases correctly rejected two indexes whose HEAD changed after a commit, emitted the exact `codexa index` recovery action, and passed after refresh. This exercises the intended fail-closed path rather than hiding staleness.

Graphify MCP interoperation

The same project configuration retained both Graphify and Codexa server blocks. Fresh final-head stdio client sessions initialized both servers, listed their tools, and made real calls:

- Graphify `graph_stats`: 26 nodes and 32 edges; no tool error.
- Codexa `change_plan`: returned `edit_ready`, saved the task snapshot, and supplied runner-aware verification and dependency checks; no tool error.

The current Graphify Python environment required an `mcp<2` constraint because of Graphify's upstream dependency compatibility. That workaround is independent of Codexa's Node runtime and is not included in the package.

Risk-Budgeted Review

Finding weights are critical 8, high 5, medium 3, and low 1. Merge requires zero critical/high findings and no more than 4 total residual points. An independent final-head review covers correctness, security, data integrity, performance bounds, compatibility, and release operations.

The first independent pass found one high-severity over-credit path: a bare `cypress run` could cover unrelated JavaScript recommendations. Commit [46104e9](#) closed the broad case. Automated PR review then found two additional gaps: Cypress files could still receive generic JavaScript credit, and a stale `.md` directory could reach file deletion logic. The final adversarial boundary review expanded this into one shared managed-artifact fix and found a medium-severity force-tracked parse-cache poisoning path. Commit [01d204f](#) closes every reproduced path with exact regressions.

Three independent final reviews covered Cypress provenance, managed-artifact boundaries, and overall risk. Their final result is 0 critical, 0 high, 0 medium, 0 low; residual score 0 and merge recommended. Static symlink/hardlink, tracked-cache poison, Windows-invalid names, MCP direct/list reads, recovery, outcome authority, stale provenance, and exact runner coverage were exercised.

Operational Watch Items (Not Scored Findings)

- Stricter classification can turn previously credited ambiguous commands into missing evidence; callers must run an explicit target or provide a state-bound verification manifest.
- Exact identity and review scans are bounded, so unusually large or actively mutating worktrees fail closed instead of claiming readiness.
- Same-user concurrent path substitution remains a theoretical operating-system boundary where Node lacks directory-handle/openat publication APIs; no static repository exploit remains.
- Session reconciliation can surface recovery warnings for damaged legacy local state rather than silently trusting it.
- Commit-pinned actions and publisher tools require deliberate maintenance updates.
- Graphify's Python MCP compatibility issue remains upstream.

Rollout and Deployment

1. Require the GitHub check, package smoke, benchmark, and native Ubuntu/macOS/Windows worktree-bootstrap lanes on the exact PR head.
2. Merge through the protected branch only after the final risk-budget review is within budget.
3. Use the repository's canonical secret-backed `release-please.yml` lane with `RELEASE_PLEASE_TOKEN` to create or update the release PR; verify its version files, changelog, and exact-head checks before merging it.
4. Let the resulting GitHub Release trigger `npm-publish.yml` with the repository's `NPM_TOKEN`; monitor npm provenance and MCP Registry publish.
5. Live-verify the GitHub tag and release, npm registry version, clean package installation, CLI version/startup, and MCP server initialization/tool call.

Rollback

Before publication, revert the feature commits and regenerate local Codexa cache/state as needed. After npm publication, do not overwrite or silently yank the immutable version: revert on `main`, publish a corrective patch, and deprecate the affected version only if its behavior is materially unsafe.